

SPEC 2111

Deep Learning and Neural Networks

Course Roadmap

PART-1: Introduction, Foundations & Essentials

- Week 1 – Introduction & Course Overview
- Week 2 – Linear Regression
 - Regression concepts, gradient descent, loss functions
- Week 3 – Logistic Regression
 - Classification, sigmoid, cross-entropy loss

PART-2: Neural Networks, Deep Learning and Training

- Week 4 – Neural Network Essentials
 - Neurons, layers, forward propagation, activation functions
- Week 5 – Training, Refining Backpropagation and Deep Neural Networks (DNNs), Backpropagation, chain rule, gradient descent optimization
- Week 6 – Preventing Overfitting, Activation Functions, Optimizers

PART-3: Specialized Architectures

- Week 7 – Convolutional Neural Networks (CNNs)
 - Convolutions, pooling, image applications
 - Assessment-1
- Week 8 – Representations & Transfer Learning
- Week 9 – Recurrent Neural Networks & LSTM
 - RNNs, LSTM/GRU, sequence modeling
- **Week 10 – Attention & Transformers** 
 - Attention mechanism, transformer architecture

PART-4: Reinforcement Learning & Projects

- Week 11 – Reinforcement Learning
 - **Assessment-2**
- Week 12 - Project Presentations & Discussions (**Project Due**)

Course Roadmap (Remaining Weeks)

# Week	Date	Topic	Notes
Week 10	22nd of April	Attention & Transformers	
Week 11	29th of April	Reinforcement Learning	ASSESSMENT 2
Week 12	6th of May	Discussions on New Topics, Project Presentations*	PROJECTS DUE 4th of May@5pm

Assessment 2 NEXT WEEK:

- **Strictly in-class; attendance will be taken**
- Time: approximately **10:05–11:00**
 - **30 questions, 50 minutes**
- Same format; only your notes are allowed

*Reach out if you cannot share the data for the project work to get a slot for presentations

Week-10 / Agenda

Theme: Attention and Transformers

- Recap on Week-9
 - RNNs/LSTMs
- Seq2Seq Modelling and Encoder-Decoder Architectures
- Attention Mechanism
- Attention Mechanism in Images
- Transformers
- Lab Notebook



Recap RNNs & LSTMs

RNNs



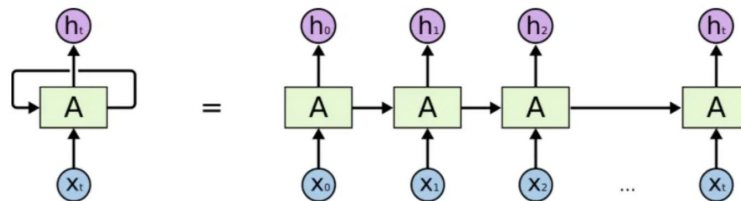
Consider a problem of **predicting the next word in a sentence**.

- Input: a sequence of words (e.g., “I went to the”)
- Output: a probability distribution over the next word (e.g., “store”, “park”, “gym”, etc.)

If we used traditional DNNs, we would first need to convert the input into a **fixed-length vector representation**, ex: using BOW or Word2Vec embeddings.

However:

- This representation **does not preserve word order**
 - model treats “I went to the store” and “store the to went I” as essentially similar representations
- As a result, model **loses the sequential structure of language**, which is crucial for meaning



RNNs are designed to handle **sequential data**

- use **recurrent connections (loops)** to allow information to persist over time
- maintain a **hidden state (memory)** that is updated at each timestep
 - hidden state carries information from previous steps forward in the sequence
- process input sequences **step-by-step in order**
- each output depends on **current input + previous hidden state**



RNN Input

- RNN input is not a single vector
 - It is a sequence of vectors for each timestamp
- Each timestep is not just one number (usually)
 - It is a set of features
- ex: Temperature readings
 - $x_t = [\text{temperature}, \text{humidity}, \text{pressure}]$
 - Each timestep has multiple features
 - Final input shape: (batch_size, timestamp, features)

Suppose we have **3 features** measured every hour:

Time	Temperature (°C)	Humidity (%)	Pressure (hPa)
t1	20	55	1012
t2	21	57	1011
t3	21	60	1010
t4	22	62	1009

Each time step is a **vector of features**:

$$x_1 = [20, 55, 1012], \quad x_2 = [21, 57, 1011], \dots$$

- Suppose we have **2 days of data**, each day has 4 hours:

Example tensor:

```
[  
  [[20,55,1012], [21,57,1011], [21,60,1010], [22,62,1009]], # Day 1  
  [[18,50,1013], [19,53,1012], [20,55,1011], [21,58,1010]] # Day 2  
]
```

- Each row = **one sequence**
- Each column along time = **one time step**
- Each innermost vector = **features at that time step**

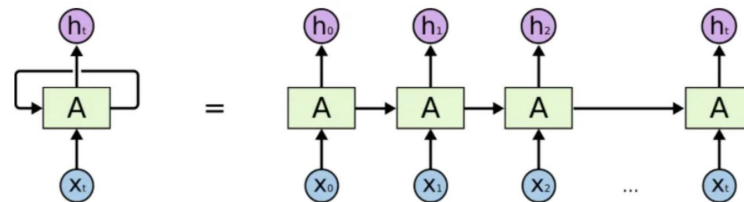
RNNs can handle variable-length sequences

RNNs can handle variable-length sequences

- because they process inputs one time step at a time using the same function repeatedly.
- At each time step: $h_t = f(x_t, h_{t-1})$
 - Same function f (same weights) is reused
 - Takes:
 - current input x_t
 - previous state h_{t-1}
 - Produces: new state h_t
- Because the model doesn't expect a fixed input size like a standard neural network
 - It just keeps applying the same transformation: $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_t$

You can:

- Stop at 3 steps or continue to 100 steps



The RNN is like a loop, not a fixed-size structure.

RNN Hidden Memory State



RNNs keep memory as they process tokens within a sequence

once a sequence ends, the memory is reset so it doesn't affect the next sequence

- Some advanced setups (stateful RNNs) can carry state across batches but this is not typical

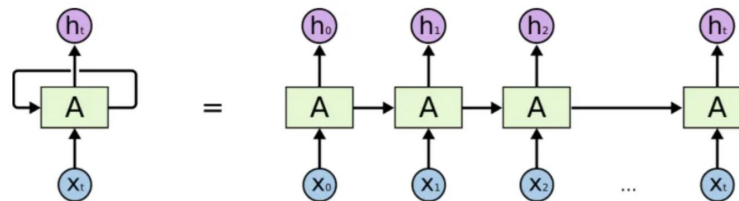
Within a sequence

- For one sequence: $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_n$
- The hidden state flows: $h_0 \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_n$
- So: Memory is preserved across time steps within the same sequence

Across different sequences (in a batch)

- If you have multiple sequences:
 - Sequence 1: "I like dogs"
 - Sequence 2: "She eats food"

Then: The hidden state is reset between sequences





RNN Setup - Definitions

- An **RNN layer** has multiple neurons, and each neuron corresponds to one dimension of the hidden state vector.
 - the layer has a hidden state vector
 - each neuron corresponds to one element of that vector
- An **RNN cell** encapsulates the computation that updates the entire hidden state vector using all neurons
 - this same computation is applied at every timestep with shared weights.
- Neurons define the size of the hidden state; the cell defines how it is updated over time.
- You can stack as many RNN layers as you want.

RNNs

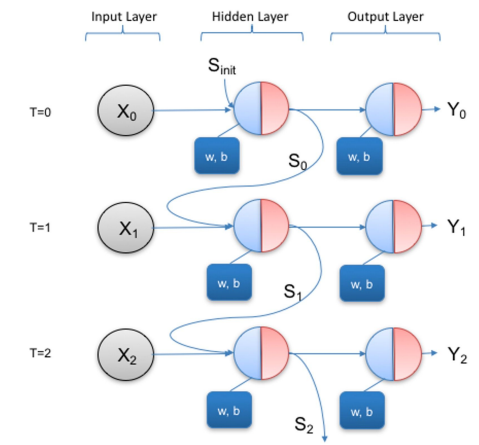


What RNNs are effective at

- processing relatively short input sequences
- capturing short-term dependencies in sequential data
 - modelling relationships where relevant information occurs within nearby time steps
 - **“how far back model needs to remember useful information”**

Limitations

- Vanishing Gradient Problem
 - even if our RNN has just one or two recurrent layers, because of the sequential nature of the input, backpropagation has to “unfold” the network over time.
 - called **backpropagation through time**
 - longer the input sequence, deeper the unfolded network.
- Handling long-term dependencies
 - as length of input sequence grows, feature vectors from the beginning of the sequence tend to be “forgotten”
- Sequential Processing:
 - Inherently processes data sequentially, leading to inefficiencies in training and inference.



Due to its limitations, plain RNNs are less common in modern applications.

Used in simple language modeling, time series prediction

LSTMs (Long Short-Term Memory Networks)



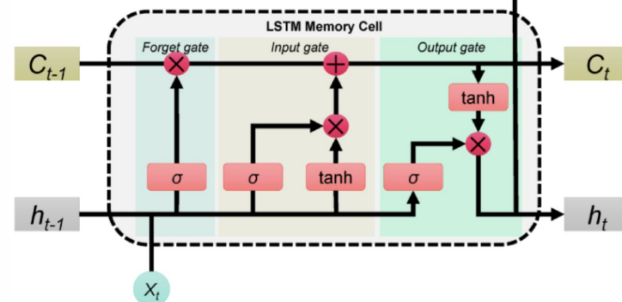
- A **gated type of RNN** designed to capture **long-term dependencies**
- Helps overcome **vanishing gradient problem** by providing a more direct path for the gradient to flow through time
- Acts like a **memory system that learns what to keep, forget, and use**

Structure

- Uses a **cell state** (long-term memory)
- Uses a **hidden state** (short-term output)
- Controlled by **three gates**:
 - **Forget gate** → what to erase
 - **Input gate** → what to store
 - **Output gate** → what to reveal
- Gates control updates, preventing excessive repeated nonlinear transformations

Intuition

- Decides what information is important early in the sequence and keeps it for later
- Allows information from early timesteps to influence later predictions



What LSTMs do well

- better at capturing long-range dependencies compared to standard RNNs.

Limitations:

- still sequential in nature, which limits parallelization.
- complexity increases with longer sequences and larger datasets, leading to higher computational costs.
- limited ability to model very long context

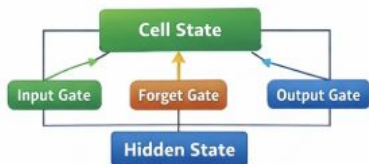
LSTM vs GRU (Gated RNNs)



LSTM vs GRU (Gated RNNs)

LSTM

(Long Short-Term Memory)



- 3 Gates: Forget, Input, Output
- Cell State + Hidden State
- More Complex, More Flexible
- Better for Long Sequences

GRU

(Gated Recurrent Unit)



- 2 Gates: Reset, Update
- Single Hidden State
- Simpler, Faster to Train
- Good for Many Tasks

Detailed Memory Control

VS

Simpler & Faster

- **GRU** is a simpler and more computationally efficient alternative to LSTM
- It was designed to:
 - reduce **model complexity**
 - reduce **number of parameters**
 - speed up training
- It often achieves:
 - **similar performance to LSTM** especially on many standard sequence tasks

Rule of thumb

- Use **LSTM** when long-term dependencies are very important
- Use **GRU** when you want **efficiency + similar performance**

Seq2Seq Modelling
Encoder-Decoder Architecture

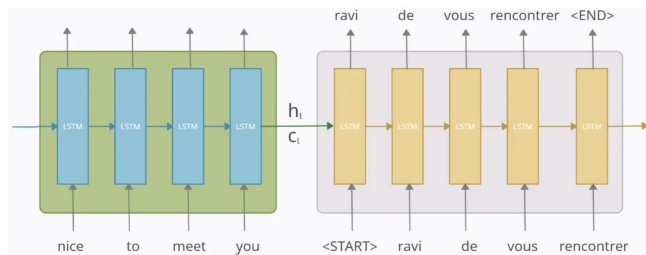
From LSTMs to Seq2Seq models

- Seq2Seq problems map one sequence to another sequence, often with different lengths
 - ex: machine translation, question answering, chatbots, and text generation.
- A standard RNN is not well suited for this setup because it predicts outputs step by step using only source tokens seen so far
 - it lacks full-sentence context
- In translation, this is a major problem because correct target words often depend on the meaning of the entire input sentence, not just the prefix read up to that point.
 - Ex: translating “The weather is nice today” into French requires understanding the full sentence before producing French translation
 - If you’re a human translator, you’d start by reading the entire source sentence before starting to translate it.

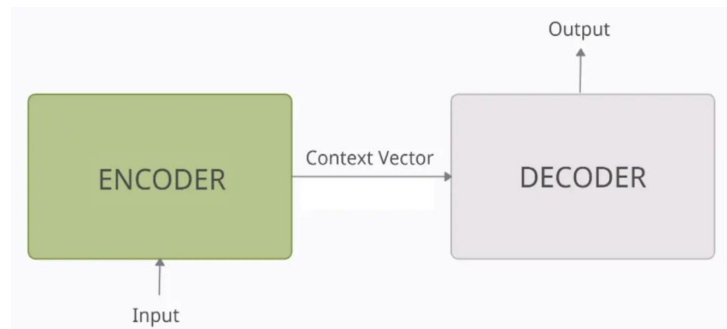
Seq2Seq solution is the Encoder-Decoder Architecture

- encoder reads the input sequence and creates an internal representation (Context Vector).
- decoder uses that representation to generate output sequence one token at a time.
 - this design better matches how human translators work: first understand the full sentence, then produce the translation.
- you can think of **encoder** as a reader and **decoder** as a writer.
 - reader understands the source sentence and passes its understanding to the writer
 - writer then produces the translation step by step.
- **other example problems to solve with Encoder Decoder architecture:**
 - part-of-speech tagging/named entity recognition (NER) → the model takes a sentence as input and outputs a corresponding sequence of labels (e.g., DET–NOUN–VERB for POS, or PERSON–ORG–LOCATION for NER)
 - text summarization (long article → short summary).
 - chatbots / dialogue systems (conversation history → reply).

Encoder Decoder Model Architecture - High Level Overview



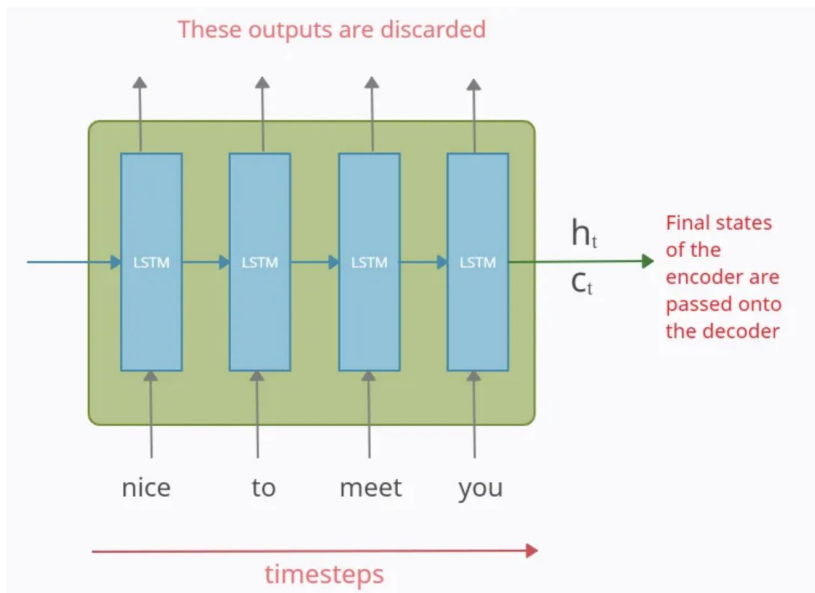
- as far as architecture is concerned, it's quite straightforward.
- model can be thought of as two LSTM cells with some connection between them.



Encoder:

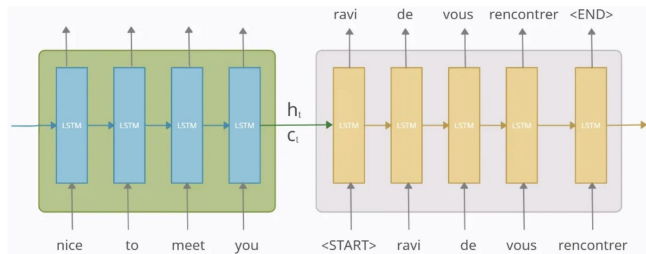
- processes each token in the input-sequence.
- tries to cram all the information about the input-sequence into a vector of fixed length **i.e. the 'context vector'**.
- **after going through all the tokens, encoder passes this vector onto the decoder.**

Encoder



- encoder is usually an LSTM that reads the input sequence one token at a time.
- at time steps $1, 2, \dots, T$
 - it processes $x_1, x_2, \dots, x_T$
 - it updates its internal hidden state h_t and cell state c_t
- As it moves through the sequence, it tries to capture “important information” from everything it has seen so far in these states.
- After the final input token, encoder’s last hidden and cell state is used as the context vector.
 - This context vector is passed to decoder, which uses it to begin generating the target sequence.
- In basic encoder-decoder model, encoder’s intermediate outputs are not passed to the decoder
 - intermediate outputs are discarded
 - only final state is used.

Decoder



- reads context vector and tries to predict target-sequence token by token.

Timestep 0 (initialisation step):

- this is the kick-off step
- input to decoder is
 - a special token: <START> (also called <SOS>)
 - encoder context (hidden + cell state)
- Then decoder produces the first real output word

Timestep 1 and beyond (generation phase):

During generation at each step:

- Input
 - previous output
 - encoder context (hidden + cell state)
- Output = next word

So:

Step 1: input = <START> → output = "ravi"

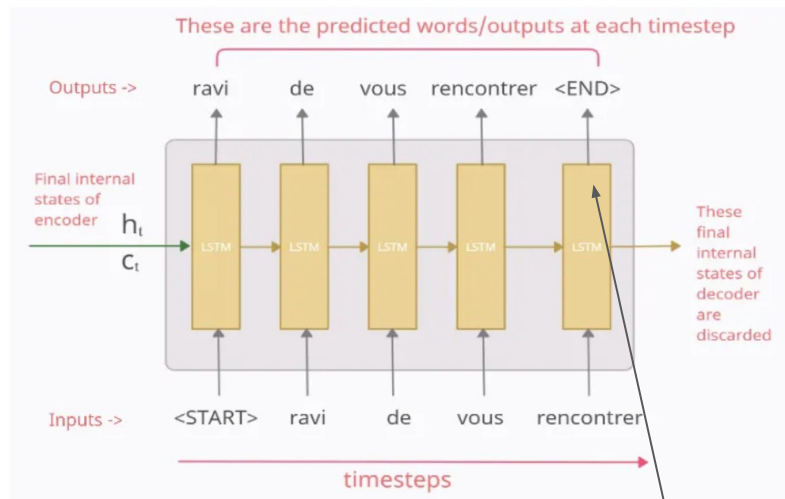
Step 2: input = "ravi" → output = "de"

Step 3: input = "de" → output = "vous"

Step 4: input = "vous" → output = "recontrer"

Step 5: input = "recontrer" → output = "END"

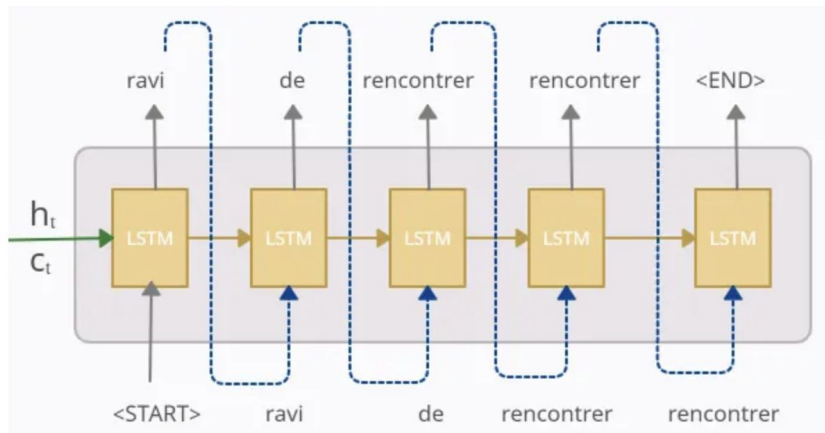
Decoder



$\langle \text{START} \rangle$ and $\langle \text{END} \rangle$ are special “control tokens” that tell the decoder where the sequence begins and where it should stop.

- decoder is also an LSTM that generates target sequence one step at a time.
- its initial states (h_0, c_0) are set to the encoder's final states (h_T, c_T)
 - this is the “one-shot” connection from encoder to decoder.
- after initialization, encoder is done!!
 - decoder runs on its own, updating its own hidden and cell states as it produces each output token.
- decoder never receives encoder's intermediate states
 - they are essentially ignored after encoder finishes.

Decoder



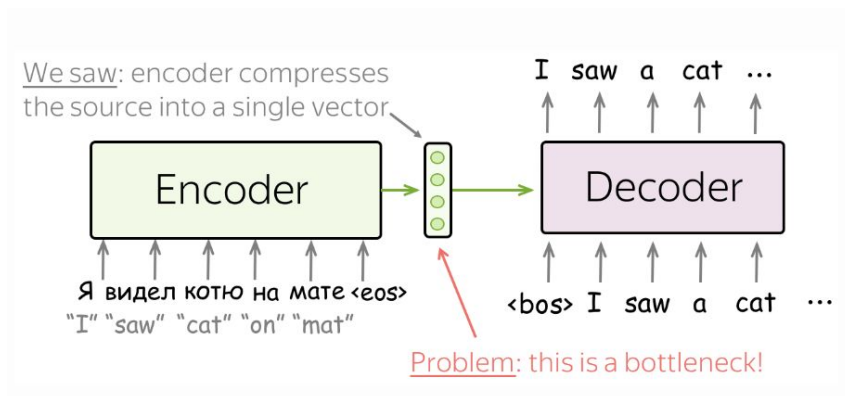
Teacher-forcing vs inference

- During training, the decoder is usually **teacher-forced**:
 - decoder's own predicted outputs are not used as input; the true target is fed at each step.
- With teacher forcing, model always starts from the **correct** previous token
 - so gradients can focus on improving the next-step prediction, not on "fixing" earlier errors
- During inference (test time), there is no ground truth, so decoder uses its own previous predictions as input to the next step.

Classic encoder–decoder model (often with RNN or LSTM) has limitations:

Limitation 1: “Fixed-size context vector” bottleneck

- leads to “Context-Loss” Problem
- entire source sequence—no matter how long or complex—is compressed into **one fixed-length vector** (e.g., 256 or 512-dim).
 - this is like taking a long story and summarizing it into a single sentence and then trying to rewrite the whole thing from that summary.
- **long or complex sentences suffer:**
 - important details at the beginning and middle can get “forgotten” because the model has limited capacity in that vector.
 - very long sequences are especially hard to handle well, since the context vector cannot faithfully represent all of them.



Ex: “The trophy doesn’t fit in the suitcase because it is too big.”

- Why RNN/LSTM struggles
 - By the time the model reaches “it,” the word “trophy” is far back
 - its information may have been weakened or lost over time.
- In longer multi-clause sentences, model may need to look far back, but earlier information can fade in the hidden state

Classic encoder–decoder model (often with RNN or LSTM) has limitations:

Limitation 2: Decoder never sees intermediate encoder states

- decoder only gets the **same fixed context** at every timestep.
- It **never directly accesses** encoder's intermediate states (i.e., the representation of each word in the input).
- This means the decoder:
 - Cannot “go back” to look at a specific word in the input at each step.
 - Has to rely on the fixed summary for the whole sentence, even when translating a word that depends on a distant part of the input.

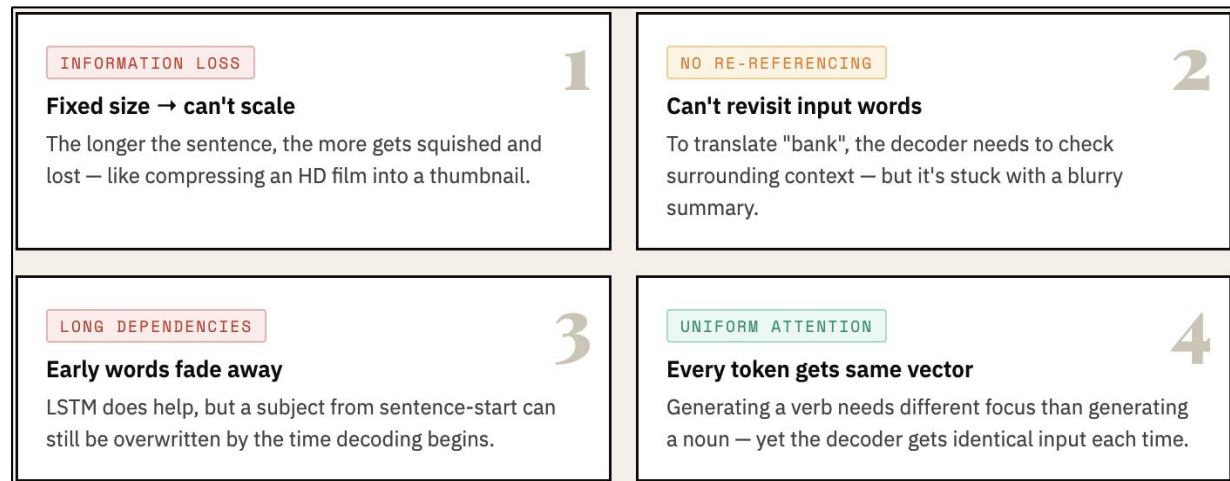
Related to Order Sensitivity: LSTMs can capture some order information by processing words sequentially:
word1→word2→word3

BUT they don't explicitly model: “Which words are related to each other?”

- **Example:** “The dog that chased the cat was fast.”
- What is *fast*? dog not cat
- **Why this is hard**
 - “dog” and “fast” are **far apart**
 - The model must: carry “dog” through many steps and *implicitly* connect it to “fast”

Limitations of Default Encoder & Decoder Models

The Fixed-Vector Bottleneck



The decoder *should focus on different parts of the input at different times* — but it always sees the same fixed summary.

This is exactly the gap that **Attention** was designed to close.

How Attention Fixes the Context Bottleneck

How Attention Fixes the Bottleneck

The Core Shift

- Old way
 - one fixed **context vector** represents the entire sentence
 - information is compressed into a single representation
- Attention
 - all encoder hidden states remain accessible
 - model can focus on different parts of the input as needed

With Attention What Happens at Each Encoder Step

- Encoder outputs many hidden states
- All hidden states are kept and passed to the decoder

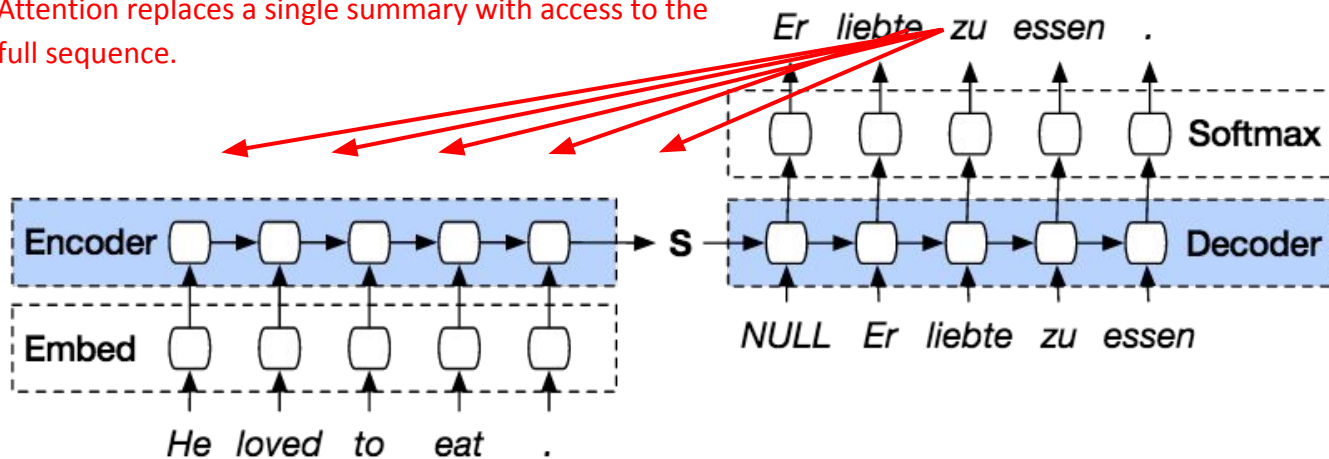
With Attention What Happens at Each Decoder Step

- decoder looks at its current state and asks: *"which input words are most relevant right now?"*
 - it scores every encoder hidden state against itself.
 - those scores become **attention weights**
 - probabilities that say *"focus 60% on word 3, 30% on word 1, 10% elsewhere."*
- a weighted sum of encoder states produces a context vector built specifically for this output word.
 - *Next word → new weights → new context vector.*
- **Result: each output word gets its own custom context**

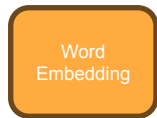
Core Idea behind Attention Mechanism

- With attention, decoder doesn't rely on a single summary of the input
- it can look back at all parts of the input sequence through hidden states and focus on what matters for each prediction

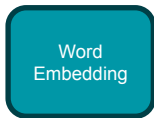
Attention replaces a single summary with access to the full sequence.



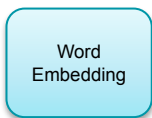
Example: Attention Mechanism



The



quick



brown



fox

- First, assume the input is represented as word embeddings.
 - Each word is converted into an embedding that captures its meaning.

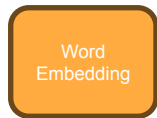
Example: Attention Mechanism

The Encoder runs first, completely, in one pass.

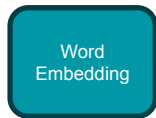
It reads the entire input sequence word by word, updating its hidden state at each step. When it's done, you have:

- A **hidden state for every input position** ($h_1, h_2, h_3 \dots h_n$)
- These are all computed and frozen before the decoder sees anything

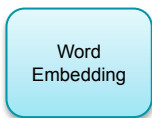
- Embeddings are fed into an encoder (e.g., LSTM), which processes the sequence and produces hidden states for each word.
- Each hidden state captures context up to that word
 - therefore the encoder summarizes the input sequence



The



quick

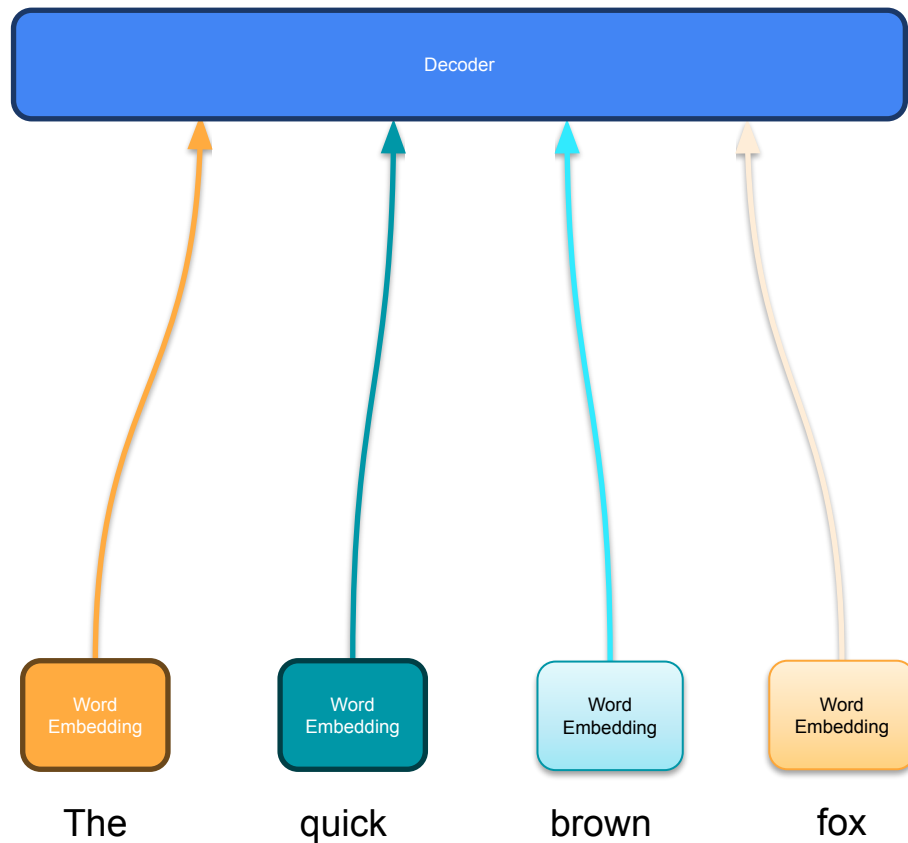


brown



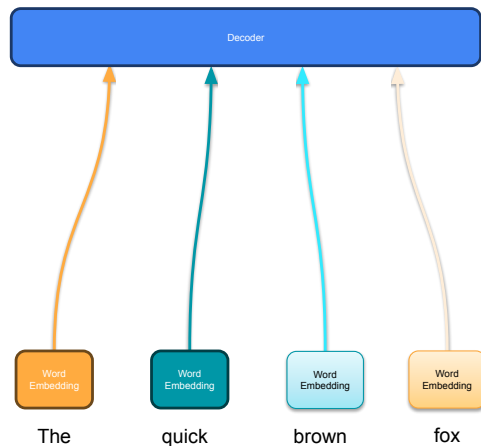
fox

Decoder without Attention



- Without attention, decoder only receives a single fixed context vector that summarizes the entire input sequence.
- This forces it to treat all parts of the input equally, even though some words are more important than others at different time steps.
- Attention addresses this by allowing the decoder to dynamically focus on the most relevant encoder hidden states when generating each output word.

Example: Decoder without Attention



Example: Pronoun Resolution

Input: "The animal didn't cross the street because **it** was too tired."

With Attention

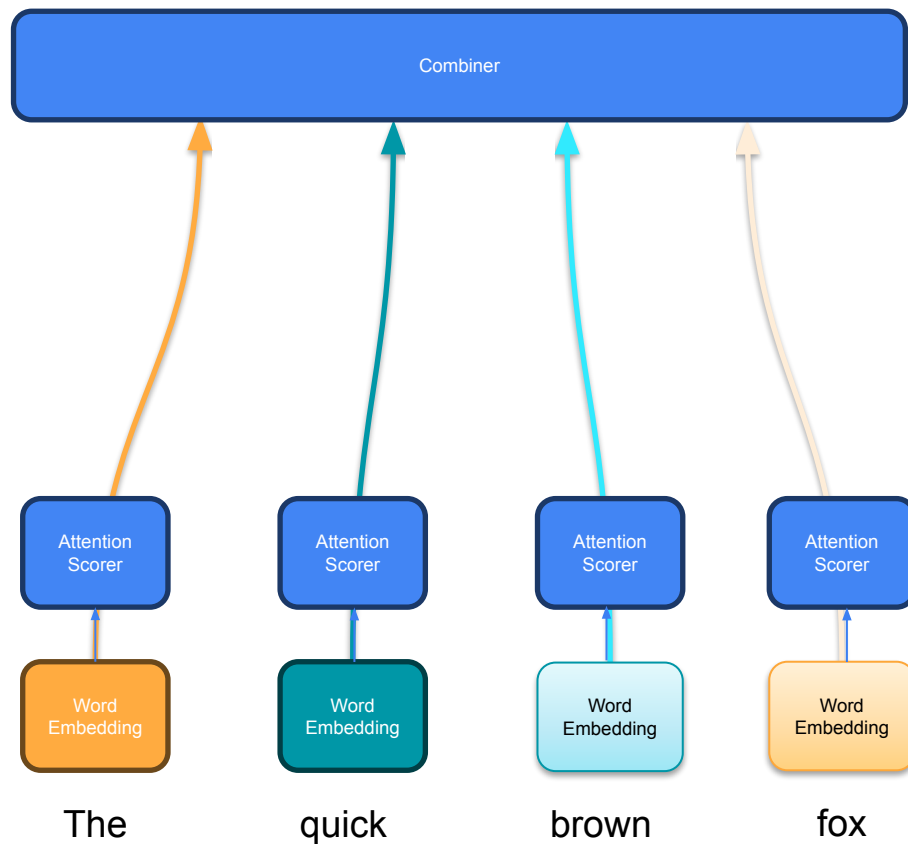
When interpreting "it":

- Model looks back at all encoder states
- Assigns higher weight to:
 - "animal"
- Lower weight to:
 - "street"

Correctly links: "it" → "animal"

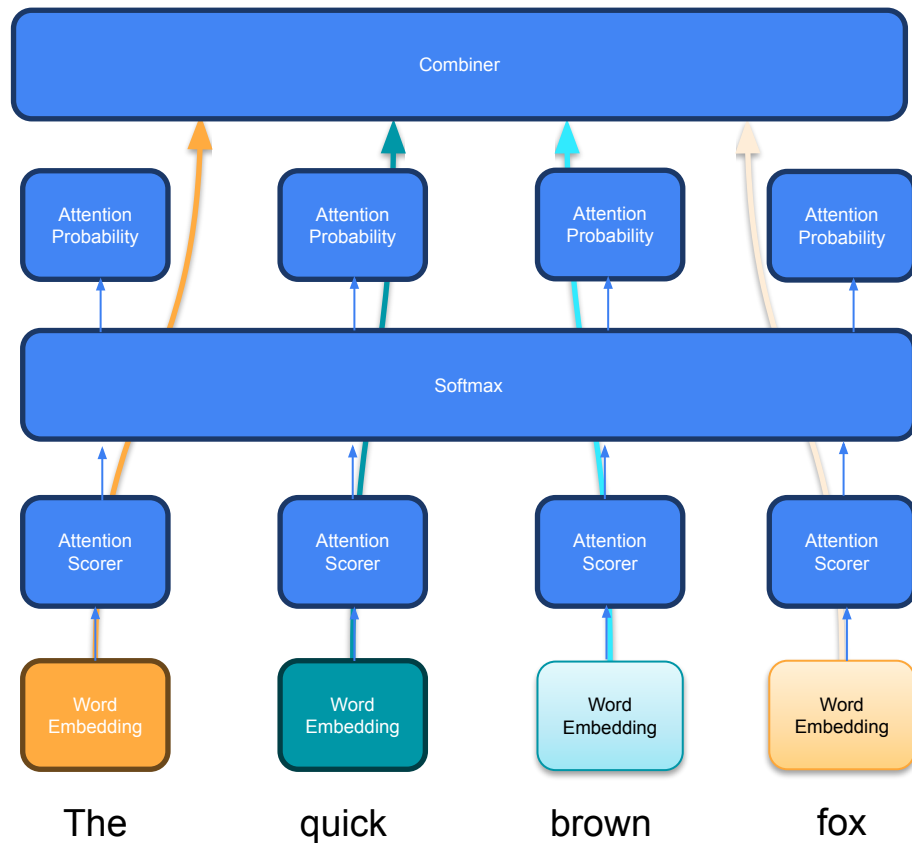
Attention helps models resolve relationships by focusing on relevant parts of the input.

The Attention Mechanism



- Attention assigns a score to each encoder hidden state for the current output step.
 - These scores reflect how relevant each part of the input is for generating the current output.
 - Higher scores indicate greater importance.
- These scores are then called **attention scores**.

Attention Probabilities Describe Importance



- each hidden state for the input words gets an attention score (how relevant it is for the current output).
- to make these scores comparable and interpretable, we pass them through a softmax layer.
- **softmax** turns the scores into probabilities that sum to 1.
 - higher probability → the word is more important
 - lower probability → the word is less important
- these probabilities tell the model how much focus to give each input element for the current prediction.
- **combiner** generates then the A weighted sum of encoder hidden states using attention probabilities.

How Attention is Implemented?

Attention as a layer:

- **Inputs:**
 - Encoder hidden states
 - Current decoder state
- **Operation:**
 - Computes relevance scores between the decoder state and each encoder hidden state
 - Applies **softmax** to obtain attention weights (sum to 1)
- **Output:**
 - A **weighted sum** of encoder hidden states
 - produces a context vector
 - focused view of the input for that current time point

At each timestep

- New attention scores are computed
- New attention weights are formed
- A new context vector is generated

You can think of it as **attention as a neural-network layer** with:

- **Inputs:** encoder hidden states, current decoder state
 - **Parameters:** learned weights that decide which words are important (attention scores),
 - **Outputs:** a **context vector** where the weights are passed through **softmax so they sum to 1**.
-
- model **looks back at all the individual hidden states**
 - so each word's representation can contribute to the final context, but the more relevant ones are emphasized.

Encoder Decoder with Attention

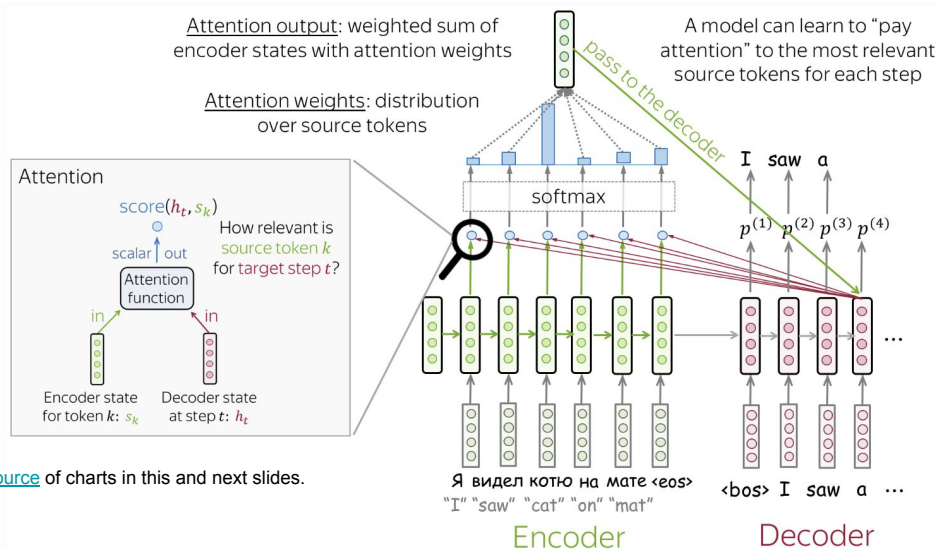
Encoder-Decoder with Attention: Flow

Encoder (runs first)

- Processes entire input sequence step by step
- Produces a sequence of hidden state
- These hidden states are **fixed and passed to the decoder**

Decoder with Attention (at each step)

- The decoder generates output one token at a time
- **At each time step**
 - uses current state + attention over encoder outputs
 - Produces the next token
 - Updates its state using the generated token
- This process repeats until an end-of-sequence token is produced

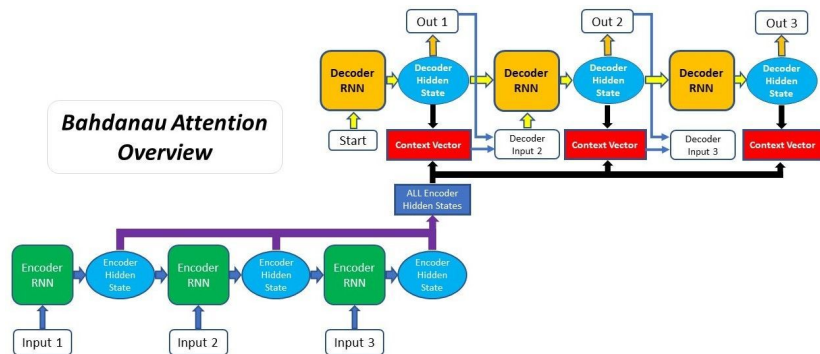


[Source](#) of charts in this and next slides.

Key Idea of Attention

- model learns **which parts of the input are most important** at each step
- Attention is fully differentiable (including softmax), the model can be trained **end-to-end**

Bahdanau Attention



- One of the first successful attention mechanisms (2014)
- Solved the context bottleneck problem in LSTM encoder-decoder models
- Introduced the idea of
 - learning where to focus

- **How it works:**

1. **Encoder:** Processes input sequence $\rightarrow$ produces hidden states $h_1, h_2, \dots, h_T$

2. **Decoder:** At each time step t :

- Uses its current hidden state s_t
- Computes **attention scores** for each encoder hidden state
- Applies **softmax** $\rightarrow$ weights (probabilities of relevance)
- Computes **context vector** = weighted sum of encoder states
- Context vector + decoder state $\rightarrow$ predict output word

- **Key benefit:**

The model can **focus on different parts of the input for each output**, instead of relying on a single compressed vector.

From Attention to Self Attention

What Attention solved first

- In encoder–decoder models, attention allows the decoder to **focus on relevant input words**
- This removes the need for a single fixed context vector
- Key idea: One sequence attends to another sequence (cross-attention)

Self-Attention idea

- Instead of one sequence attending to another:
 - each word attends to **all other words in the same sequence**
- Every token:
 - looks at all tokens
 - decides what is relevant
 - builds its own contextual meaning
- This shifts from fixed embeddings to build contextual embeddings

Example: Word Embedding → Contextual Embedding (Self-Attention)

Sentence: “The animal didn’t cross the street because it was tired.”

1. Initial word embeddings (non-contextual)

Each word starts as a **fixed vector**, independent of

- “animal” → x_{animal}
- “it” → x_{it}
- “street” → x_{street}

At this stage:

- “it” has the same embedding in every sentence
- no understanding of meaning in context

Example: Word Embedding → Contextual Embedding (Self-Attention)

Sentence: “The animal didn’t cross the street because it was tired.”

1. Initial word embeddings (non-contextual)

Each word starts as a **fixed vector**, independent of

- “animal” → x_{animal}
- “it” → x_{it}
- “street” → x_{street}

At this stage:

- **“it” has the same embedding in every sentence**
- no understanding of meaning in context

2. Self-attention step (what happens)

When processing for word “it”:

Self-attention asks: “Which words in this sentence are relevant to ‘it’?”

It computes attention weights like:

- animal → 0.7 (very important)
- street → 0.1 (less relevant)
- tired → 0.2 (some relevance)

3. Weighted mixing

Instead of staying the same, “it” is updated:

$$y_{it} = 0.7 \cdot V_{animal} + 0.1 \cdot V_{street} + 0.2 \cdot V_{tired}$$

Example: Word Embedding → Contextual Embedding (Self-Attention)

Sentence: “The animal didn’t cross the street because it was tired.”

1. Initial word embeddings (non-contextual)

Each word starts as a **fixed vector**, independent of

- “animal” → x_{animal}
- “it” → x_{it}
- “street” → x_{street}

At this stage:

- “it” has the same embedding in every sentence
- no understanding of meaning in context

2. Self-attention step (what happens)

When processing for word “it”:

Self-attention asks: “Which words in this sentence are relevant to ‘it’?”

It computes attention weights like:

- animal → 0.7 (very important)
- street → 0.1 (less relevant)
- tired → 0.2 (some relevance)

3. Weighted mixing

Instead of staying the same, “it” is updated:

$$y_{it} = 0.7 \cdot V_{animal} + 0.1 \cdot V_{street} + 0.2 \cdot V_{tired}$$

4. New result: contextual embedding

Now:

- “it” is no longer just a generic vector
- it becomes a **context-aware representation**

New meaning: “it” now strongly encodes that it refers to “animal”

Self-attention transforms static word embeddings into contextual embeddings by mixing information from all relevant words in the sentence.

From standalone word embeddings to contextual embeddings

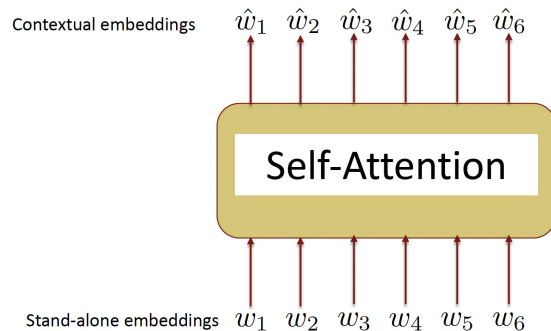
Self-attention is a layer, similar in spirit to attention.

Inputs: A sequence of token embeddings

For each word:

- Computes relevance scores between **each word and every other word in the same sequence**
- Applies **softmax** over scores
 - produces attention weights (sum to 1)
 - Softmax turns similarity scores into “how much to focus on each word”, and the model uses those weights to mix information from all words.

Output: A sequence of contextual embeddings

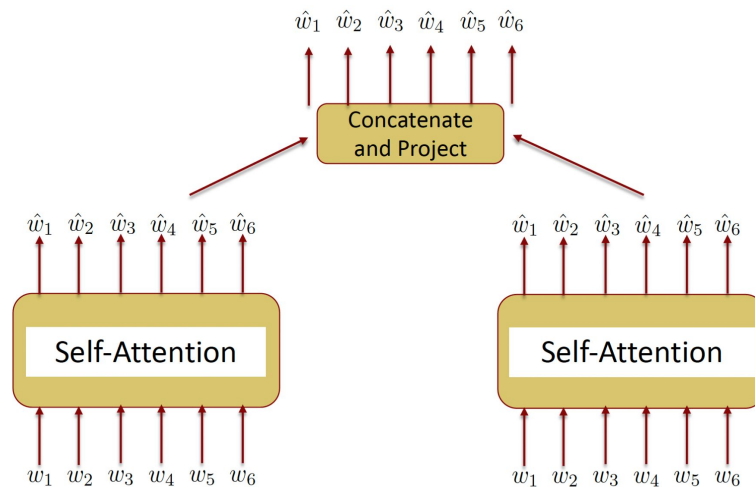


After Self-Attention

- Each word now has a context-aware representation.
- These representations are then used for:
 - prediction (next word in a sentence)
 - classification (e.g., sentiment)
 - generation (e.g., translation, text generation)

From One Attention Head to Multi-Head Attention

- Attention can contain multiple attention heads.
 - Each attention head is like a small independent module that learns a specific type of relationship.
 - takes the input
 - applies its own learned transformations,
 - computes attention weights, and
 - produces an output.
- Instead of learning **one single way of looking at a sentence**, multi-head attention lets the model learn **multiple types of relationships in parallel**.
- Each head focuses on a different pattern in the data.



From One Attention Head to Multi-Head Attention

Why this is needed

Language contains many simultaneous relationships, for example:

- grammatical structure (subject–verb agreement)
- tense (past / present / future)
- tone or sentiment
- entity relationships (who refers to whom)

A single attention mechanism may not capture all of these well at once.

What each attention head does

- Each head is like an independent attention module
- It:
 - takes the same input embeddings
 - applies its own learned transformations
 - computes its own attention weights
 - produces its own contextual output

So each head may focus on different relationships, e.g.:

Head 1 → syntax (who did what)

Head 2 → meaning (semantic similarity)

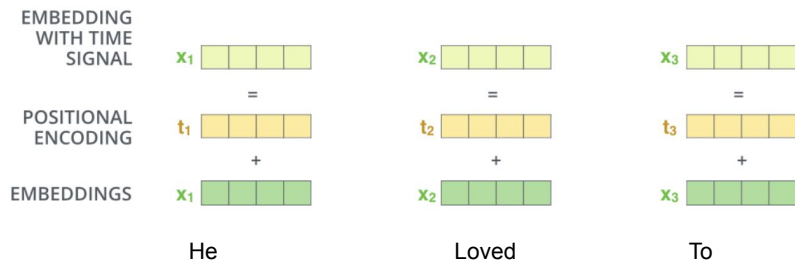
...

Think of it like:

- Single attention head → one “perspective” on the sentence
- Multi-head attention → multiple experts looking at the sentence in different ways simultaneously

Similar to: convolutional layers using multiple filters to detect different patterns

Taking Order Account : Position Encoding



- so far, attention itself is order-agnostic—it doesn't know which word comes first, second, etc.
- but word order can still be important for translation as an input dimension to consider

To incorporate word order, we use **positional encoding**.

- For each word, add a vector that represents its position in the sequence to its word embedding.
 - This means that each input embedding is the sum of two components:
 - the standard word embedding (capturing meaning)
 - a positional embedding (capturing position in the sentence)
- As a result, model receives input embeddings that contain both **semantic information** and **positional information**.
 - This allows model to understand and use order of words in a sentence.

Without positional encoding, the model would see the sentence as a bag of words — positional encoding gives it a sense of order.

Attention for Image Processing

Examples of Attention in Image Processing



Attention is a **general mechanism for selecting relevant information**, so it is used in many types of tasks

Just like in language, attention in images lets the model 'look' at the most important parts of the input before making a prediction.

Example Task: Generate a descriptive caption from an image (e.g., "Several people sitting in a park")

How Attention Works:

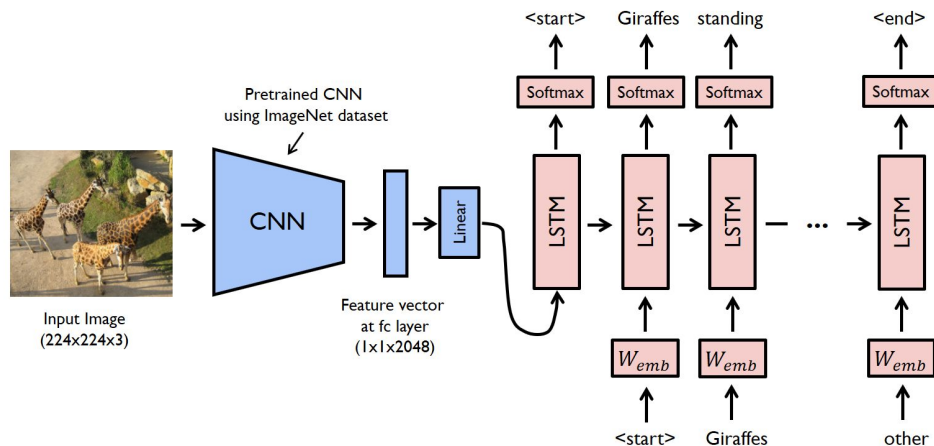
1. CNN extracts feature maps from the image.
2. At each word prediction step, attention mechanism focuses on relevant regions (e.g., the bench, people, trees).
3. decoder generates next word based on both previous words and attended image regions.

Image Captioning (An Encoder Decoder Problem)

Training Data for Image Captioning

- Input: Image (e.g., giraffes in a park)
- Target Output: Corresponding caption (text), e.g., "3-4 giraffes standing in a grassy field"

Every image is paired with one or more human-written captions.



How Training Works

CNN Encoder: Extracts feature maps from the image.

LSTM Decoder: Generates caption one word at a time.

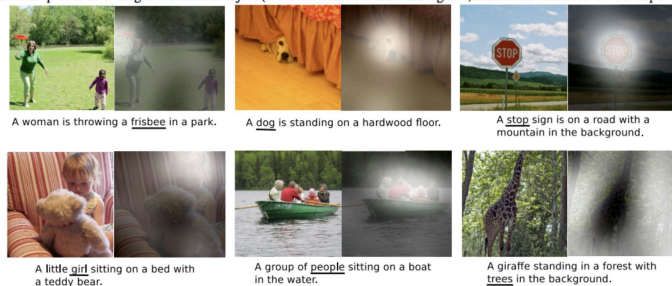
Attention: At each word step, decoder focuses on relevant image regions.

Loss Function: Compares predicted words vs. ground truth words (cross-entropy) → backpropagate through CNN + LSTM + attention.

Image Captioning (An Encoder Decoder Problem)

- Encoder is CNN
- Decoder is LSTM with Attention
- LSTM decoder generates caption word by word
- At each time step t :
 - Uses previous word (embedded as a vector) as input
 - Computes attention over feature vectors
 - Produces context vector = weighted sum of features
 - Combines
 - context vector + LSTM hidden state
 - Predicts next word
- Even though the words are not part of the image, decoder learns to map image features to words from training data during backpropagation

Figure 3. Examples of attending to the correct object (white indicates the attended regions, underlines indicated the corresponding word)



Transformers: Attention is all you need

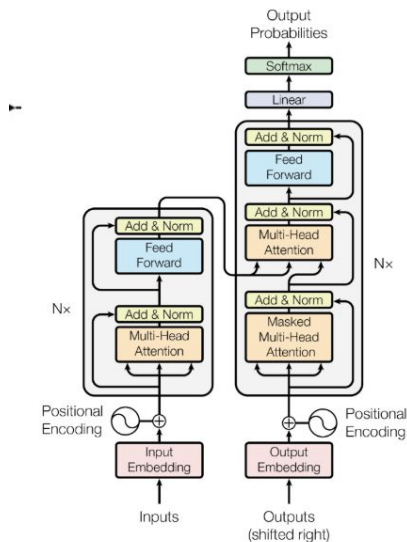
What if we build an entire model using only this idea of attention?

Transformers

- self-attention allows a model to build context-aware representations
 - by letting each word interact with all other words in a sequence.
- Transformer architecture builds an entire model on the idea of self attention
- Before Transformers, different architectures had different core ideas:
 - CNNs → local feature extraction using filters
 - LSTMs / RNNs → sequential processing with memory
 - But both have a limitation: information flow is either local or sequential
- Transformers take a different approach: Make self-attention the core building block of the entire model.
- Core idea is simple
 - model can look at the entire input at once and decide what's important for each prediction.

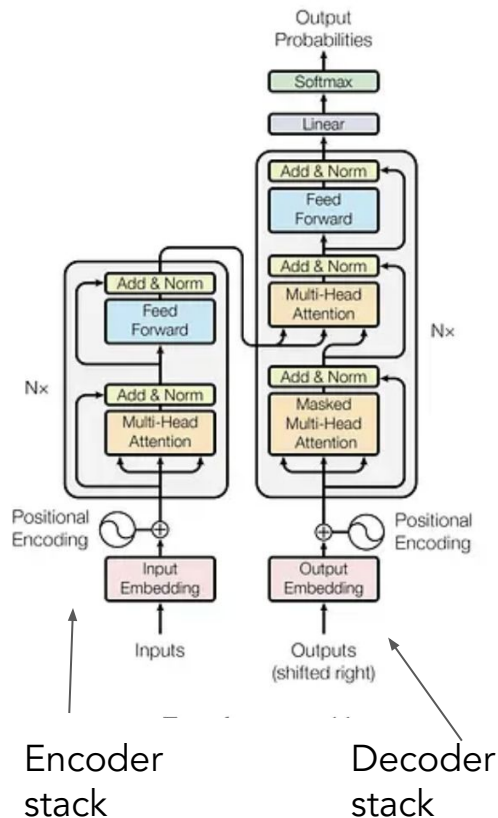
A Transformer is a Model Architecture

Transformers



- a transformer is a model architecture
- it essentially includes:
 - multiple **self-attention layers**
 - **feed-forward networks** (for non-linearity and transformation)
 - **positional encoding** (to inject word order)
- it processes entire sequence at once (in parallel)

The (Original) Transformer Architecture



Main Components

- Encoder-Decoder Structure
 - Encoder reads and processes the input sequence
 - Decoder generates the output sequence
- Each layer contains:
 - Self-Attention
 - Looks at all tokens in the sequence
 - Learns relationships between them
- Feedforward Network (Dense layer)
 - Processes each position independently

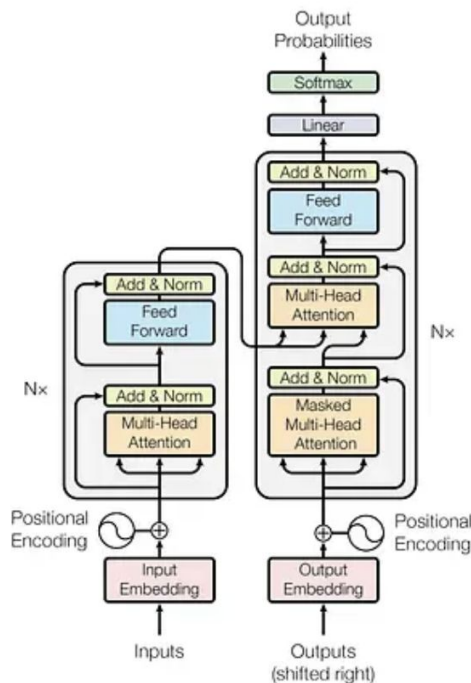
Key Differences from LSTM

- No step-by-step processing
- Entire sequence processed **at once**
- Attention replaces recurrence

Transformers: Step-by-Step Example

Example Task: Translation

- Input: “I like dogs”
- Output: “Er mag Hunde”



Step 1: Input Processing (Encoder Input)

Sentence:

I like dogs

Convert to vectors:

- Each word → **embedding**
- Add **positional encoding**

[I] [like] [dogs]

↓ ↓ ↓
vectors + position

Now we have a matrix of word representations

Step 2: Encoder – Self-Attention

What happens:

Each word looks at **all other words**

Example:

- “like” attends to:
 - “I” (who likes)
 - “dogs” (what is liked)
- Compute **attention scores**
- Convert to weights (softmax)
- Weighted combination of all words

Output: Each word becomes **context-aware**

Result:

[I*] [like*] [dogs*]

(now enriched with context)

Step 3: Encoder Output

- Final encoder output = **contextual representations**
- Passed to decoder

Transformers: Step-by-Step Example

- Decoder -

Step	Decoder sees	Should predict
1	<start>	Er
2	<start> Er	mag
3	<start> Er mag	Hunde

Decoder is trained to **predict next word**, so it must only see **previous words**, not current or future ones.

Step 5: Decoder – Two Types of Attention

1. Self-Attention (within output)

- Decoder looks at: <start> Er mag
- Understands what has been generated so far

2. Encoder–Decoder Attention (“normal” attention)

- Decoder looks at: Encoder output (I like dogs)
- Aligns:
 - “Er” ↔ “I”
 - “mag” ↔ “like”
 - “Hunde” ↔ “dogs”
- Alignment means model learns which input words are most relevant for each output word.
 - Attention weights capture this by assigning higher importance to corresponding input words.

Example Task: Translation

- Input: “I like dogs”
- Output: “Er mag Hunde”

Step 6: Training (Teacher Forcing)

- Always feed **previous word**
- Compare prediction vs target
- Compute loss → backprop

The Transformer's Novelty and Mechanisms

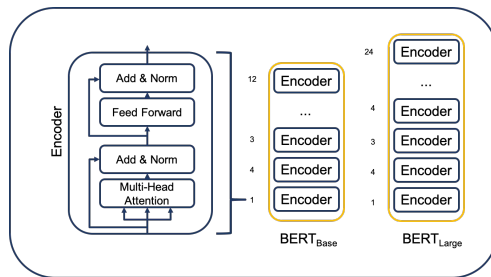
- **Encoder–decoder structure for sequence tasks**
 - Same high-level idea as LSTM seq2seq
 - **Encoder** → understands input sentence
 - **Decoder** → generates output sentence
- **Processes all words in parallel (no recurrence)**
 - Big difference from LSTMs
 - Instead of reading: I → like → dogs
 - Transformer processes: [I] [like] [dogs] (all at once)
 - Faster and more efficient
 - No step-by-step dependency
- **Positional encoding preserves word order**
 - Since there's no sequence order processing, we must add position manually
 - Add position info to each word:
 - [I + pos1] [like + pos2] [dogs + pos3]
 - Model knows: “I” comes first, “dogs” comes last
- **Self-Attention Mechanism**
 - Each word looks at all other words
 - Learns relationships like: “like” ↔ “I”, “like” ↔ “dogs”
 - Captures short-term AND long-term dependencies

Beyond the Original Transformer: BERT, GPT, ChatGPT

- BERT is essentially just the encoder part of the Transformer, repeated many times to build a deep model that understands language

What is BERT doing?

- Takes input sentence: "I like dogs"
- Passes it through **multiple encoder layers**
- Each layer:
 - Applies **self-attention**
 - Builds deeper understanding
- Final output = **context-aware representation of each word**
- **Use BERT when you want the model to understand text and make decisions about it—like classification, answering questions, or finding meaning—not when you want to generate new text.**



BERT is for understanding text;
GPT is for generating text.

Example Task with Bert: Find an answer inside a passage

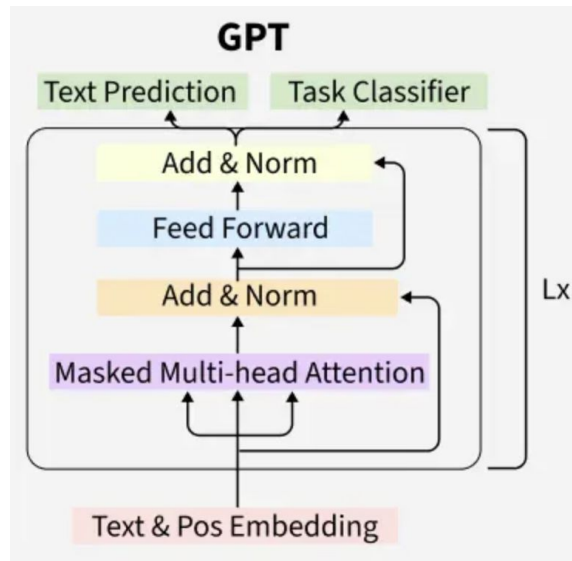
Example:

Passage: "Paris is the capital of France", Question: "What is the capital of France?" Output: "**Paris**"

BERT identifies the **relevant span of text**

Beyond the Original Transformer

GPT Series (GPT-3, GPT-4, GPT-5, etc.)



- **Type:**
 - Transformer **decoder-only model** (causal language model)
- **Key idea:**
 - Predicts **the next word in a sequence**
- GPT is basically a large stack of Transformer decoder layers, trained on huge amounts of data to generate text.
- **Use case:** text generation, summarization, code generation, etc
- **Example:**
 - Prompt: "Once upon a time,"
 - GPT predicts: "there was a dragon who lived in a cave."

Lab Work

- Check the notebook and go through it to see practical examples and review the theory as well.

Appendix

Timeline in a Nutshell

- **~2014–2015:** The first **attention mechanisms** were proposed for **RNN / encoder–decoder** models in machine translation
 - Attention was added **on top of** LSTMs/RNNs to let decoder selectively focus on different parts of the input at each step, rather than relying only on a fixed context vector.
- **2017: Transformer** architecture was introduced in the “*Attention Is All You Need*” paper.
 - it **dropped RNNs entirely** and used **self-attention** as the main building block.
 - therefore attention is the older idea
 - transformers are the architecture that uses attention in a new, highly parallel way.

Attention Is All You Need

[Article](#) [Talk](#)

From Wikipedia, the free encyclopedia

"**Attention Is All You Need**"^[1] is a 2017 research paper in [machine learning](#) authored by eight scientists working at [Google](#). The paper introduced a new [deep learning](#) architecture known as the [transformer](#), based on the [attention mechanism](#) proposed in 2014 by Bahdanau *et al.*^[2] The transformer approach it describes has become the main architecture of a wide variety of artificial intelligence, including [large language models](#).^{[3][4]} At the time, the focus of the research was on improving [Seq2seq](#) techniques for [machine translation](#), but the authors go further in the paper, foreseeing the technique's potential for other tasks like [question answering](#) and what is now known as [multimodal generative AI](#).^[1]

Using BERT & GPT with Transformers

Using Hugging Face Transformers

- Hugging Face Transformers is an open-source Python library that provides a unified interface to state-of-the-art machine learning models
- Provides pretrained models like:
 - BERT (understanding)
 - GPT (generation)

```
from transformers import pipeline
```

```
# BERT-style task (classification)
classifier = pipeline("sentiment-analysis")
print(classifier("I love this course!"))
# Output: [{'label': 'POSITIVE', 'score': 0.9998}]
```

```
# GPT-style task (generation)
generator = pipeline("text-generation", model="gpt2")
print(generator(
    "Once upon a time",
    max_length=20
))
# Output: [
#   {
#     'generated_text': 'Once upon a time, there was a young
# princess who lived in a castle...'
#   }
# ]
```

Automatically load a **BERT-like model** (or similar fine-tuned model)

